

INFORME FINAL DEL PROYECTO EVENTRA

1. Información General

Nombre del proyecto: Eventra

Autor: Daniel Alejandro Martínez Escandón

Tipo de proyecto: Aplicación móvil Android con arquitectura de microservicios

Tecnologías principales: Android Studio, Java, XML, Node.js, Express.js y PostgreSQL

Despliegue: Render

Repositorio: <https://github.com/Danma05/eventra-app>

Eventra es una aplicación móvil orientada a la gestión de eventos deportivos de running. La plataforma permite que los organizadores creen y administren eventos, gestionen participantes, controlen carreras y publiquen rankings. Los corredores pueden registrarse, iniciar sesión, consultar eventos, inscribirse, participar en carreras y visualizar resultados publicados.

2. URL del Repositorio GitHub

El código fuente completo del proyecto se encuentra alojado en GitHub:

<https://github.com/Danma05/eventra-app>

El repositorio incluye:

Aplicación móvil Android.

Microservicios backend.

Scripts SQL.

Documentación técnica.

Manual de usuario.

Manual técnico.

Modelo relacional.

Credenciales de prueba.

Instrucciones de ejecución.

APK de la aplicación.

3. URL de la Aplicación Desplegada

Eventra no corresponde a una aplicación web tradicional con una interfaz frontend web desplegada. El proyecto corresponde a una aplicación móvil Android que consume microservicios REST desplegados en Render.

La aplicación Android se encuentra dentro del repositorio y puede ejecutarse desde Android Studio o mediante el APK incluido.

Los servicios desplegados en Render son los siguientes:

Microservicio	URL
Auth Service	https://eventra-auth.onrender.com
User Service	https://eventra-user.onrender.com
Events Service	https://eventra-events.onrender.com
Registrations Service	https://eventra-registrations.onrender.com
Results Service	https://eventra-results.onrender.com
Activity Service	https://eventra-activity.onrender.com

4. Nota sobre Render y el Plan Gratuito

Los microservicios de Eventra se encuentran desplegados en Render utilizando el plan gratuito. Debido a esta modalidad, los servicios pueden suspenderse temporalmente después de un periodo de inactividad.

Cuando se consume un endpoint después de varios minutos sin uso, el primer llamado puede tardar entre 30 segundos y 2 minutos aproximadamente, mientras Render reactiva el servicio. Después de esta activación inicial, las siguientes solicitudes responden normalmente.

Este comportamiento no representa un error de la aplicación ni de los microservicios. Es una característica propia del entorno gratuito utilizado para el despliegue académico del proyecto.

5. Descripción General del Proyecto

Eventra busca facilitar la organización, administración y participación en eventos deportivos de running mediante una solución móvil conectada a servicios backend desplegados en la nube.

La aplicación permite gestionar el ciclo principal de un evento deportivo:

Registro e inicio de sesión de usuarios.
Diferenciación de roles entre organizador y corredor.
Creación y administración de eventos.
Inscripción de corredores.
Consulta de participantes.
Control de carrera.
Seguimiento de actividades.
Finalización de eventos.
Publicación de rankings.
Visualización de resultados.

6. Arquitectura General

El proyecto implementa una arquitectura basada en microservicios REST. La aplicación Android consume directamente los servicios desplegados en Render mediante peticiones HTTP y respuestas en formato JSON.

La estructura general es:

Aplicación Android

↓

Microservicios REST en Render

↓

Bases de datos PostgreSQL

Cada microservicio tiene una responsabilidad específica y trabaja con su propia base de datos, permitiendo mayor organización, mantenibilidad y separación de responsabilidades.

7. Microservicios Implementados

7.1 Auth Service

Puerto local: 3001

URL Render: <https://eventra-auth.onrender.com>

Base de datos: eventra_auth

El Auth Service se encarga de la autenticación de usuarios, registro, inicio de sesión y generación de tokens JWT.

Método	Endpoint	Descripción
POST	/auth/register	Registra un nuevo usuario en el sistema
POST	/auth/login	Permite iniciar sesión y retorna token JWT

7.2 User Service

Puerto local: 3002

URL Render: <https://eventra-user.onrender.com>

Base de datos: eventra_user

El User Service administra la información del perfil de usuario, datos personales y datos asociados al corredor.

Método	Endpoint	Descripción
POST	/users/profile	Crea el perfil del usuario autenticado
GET	/users/profile/me	Consulta el perfil del usuario autenticado
PUT	/users/profile/me	Actualiza el perfil del usuario autenticado
GET	/users/profile/:authUserId	Consulta perfil por identificador de autenticación

7.3 Events Service

Puerto local: 3003

URL Render: <https://eventra-events.onrender.com>

Base de datos: eventra_events

El Events Service administra los eventos deportivos y el control general de carrera. Permite crear, editar, eliminar, consultar eventos, definir rutas, iniciar, pausar, reanudar y finalizar carreras.

Método	Endpoint	Descripción
GET	/events	Lista todos los eventos disponibles
GET	/events/:id	Consulta el detalle de un evento

Método	Endpoint	Descripción
POST	/events	Crea un nuevo evento
PUT	/events/:id	Actualiza un evento
DELETE	/events/:id	Elimina un evento
GET	/events/organizer/my	Lista los eventos creados por el organizador autenticado
PUT	/events/:id/route	Define o actualiza la ruta del evento
POST	/events/:id/start	Inicia una carrera
POST	/events/:id/pause	Pausa una carrera
POST	/events/:id/resume	Reanuda una carrera
POST	/events/:id/finish	Finaliza una carrera

7.4 Registrations Service

Puerto local: 3004

URL Render: <https://eventra-registrations.onrender.com>

Base de datos: eventra_registrations

El Registrations Service administra la inscripción de corredores, participantes por evento y conteos de inscripciones.

Método	Endpoint	Descripción
POST	/registrations	Inscribe un usuario a un evento
GET	/registrations/my	Consulta las inscripciones del usuario
POST	/registrations/counts	Consulta conteos de inscritos por evento
GET	/registrations/my/counts	Consulta conteos de eventos del organizador
GET	/registrations/event/:eventId/participants	Lista participantes inscritos en un evento
GET	/registrations/check	Verifica si un usuario está inscrito en un evento

7.5 Results Service

Puerto local: 3005

URL Render: <https://eventra-results.onrender.com>

Base de datos: eventra_results

El Results Service administra los resultados deportivos, rankings y publicación de clasificaciones oficiales.

Método	Endpoint	Descripción
POST	/results	Registra un resultado
GET	/results/events/published	Lista eventos con resultados publicados
GET	/results/event/:eventId	Consulta resultados publicados por evento
GET	/results/event/:eventId/drafts	Consulta resultados preliminares
PUT	/results/event/:eventId/publish	Publica oficialmente el ranking de un evento

7.6 Activity Service

Puerto local: 3006

URL Render: <https://eventra-activity.onrender.com>

Base de datos: eventra_activity

El Activity Service administra las actividades deportivas, ubicación GPS, sesiones de carrera, participantes activos y ranking en vivo.

Método	Endpoint	Descripción
POST	/activities/start	Inicia una actividad deportiva
POST	/activities/location	Registra ubicación GPS del corredor
POST	/activities/finish	Finaliza una actividad
POST	/activities/pause	Pausa una actividad
POST	/activities/resume	Reanuda una actividad
GET	/activities/event/:eventId/active	Consulta participantes activos en un evento

Método	Endpoint	Descripción
GET	/activities/event/:eventId/live-ranking	Consulta ranking en vivo del evento
POST	/activities/event/:eventId/finish-open-sessions	Finaliza sesiones abiertas de un evento

8. Patrones de Diseño y Buenas Prácticas Implementadas

8.1 Arquitectura de Microservicios

El backend está dividido en microservicios independientes. Cada servicio tiene una responsabilidad definida y una base de datos propia. Esta arquitectura facilita el mantenimiento, la escalabilidad y la separación lógica del sistema.

Microservicios implementados:

- Auth Service.
- User Service.
- Events Service.
- Registrations Service.
- Results Service.
- Activity Service.

8.2 Arquitectura Cliente-Servidor

La aplicación Android actúa como cliente móvil. Los microservicios desplegados en Render funcionan como servidor backend. La comunicación se realiza mediante APIs REST usando JSON como formato de intercambio.

8.3 Separación por Capas en Backend

Los microservicios utilizan una organización por capas:

- Routes.
- Controllers.
- Services.

- Models.
- Config.
- Middlewares.

Esta separación permite que cada archivo tenga una responsabilidad específica y evita concentrar toda la lógica en un solo punto.

8.4 Patrón Adapter en Android

La aplicación Android utiliza adaptadores para mostrar listas mediante RecyclerView.

Ejemplos:

- EventAdapter.
- OrganizerEventAdapter.
- RegisteredEventAdapter.
- ResultAdapter.
- ActiveParticipantAdapter.

Esto permite separar la lógica de visualización de listas respecto a las actividades principales.

8.5 Gestión de Sesión

La sesión del usuario se administra mediante SharedPreferences en la clase SessionManager.

Se almacenan datos como:

- Token JWT.
- Correo del usuario.
- Rol del usuario.
- Estado de sesión.

8.6 Centralización de URLs

Las URLs de los servicios backend se administran desde la clase ApiConfig.java. Esto permite cambiar fácilmente las rutas de los microservicios sin modificar todas las pantallas de la aplicación.

8.7 Validación de Conectividad

La aplicación utiliza NetworkUtils.java para validar conexión a internet antes de realizar operaciones dependientes de red. Esto mejora la experiencia del usuario y evita errores inesperados.

8.8 Manejo de Errores

La aplicación implementa manejo de excepciones mediante bloques try/catch y registro de errores con Log.e(). Esto permite identificar fallos de conexión, errores de respuesta del servidor y problemas de procesamiento.

9. Pruebas Realizadas a un Microservicio

Para la validación del backend se realizaron pruebas sobre el Auth Service utilizando Thunder Client.

9.1 Prueba de Registro de Usuario

Endpoint:

POST <https://eventra-auth.onrender.com/auth/register>

Body utilizado:

```
{  
  "email": "runner.demo@eventra.com",  
  "password": "EventraRunner2026*",  
  "account_type": "RUNNER"  
}
```

Resultado esperado:

- Código HTTP 201 Created.
- Usuario registrado correctamente.
- Retorno de datos básicos del usuario.

9.2 Prueba de Inicio de Sesión

Endpoint:

POST <https://eventra-auth.onrender.com/auth/login>

Body utilizado:

```
{  
"email": "runner.demo@eventra.com",  
"password": "EventraRunner2026*"  
}
```

Resultado esperado:

- Código HTTP 200 OK.
- Retorno de token JWT.
- Retorno de datos del usuario autenticado.
- Acceso posterior a funcionalidades protegidas de la aplicación.

10. Credenciales de Prueba

Las credenciales principales para evaluación son:

Usuario Organizador

Correo:

organizador.demo@eventra.com

Contraseña:

EventraOrg2026*

Funciones disponibles:

Crear eventos.

Editar eventos.

Consultar eventos propios.

Ver participantes.

Controlar carrera.

Publicar ranking.

Usuario Corredor

Correo:

runner.demo@eventra.com

Contraseña:

EventraRunner2026*

Funciones disponibles:

Consultar eventos.

Inscribirse a eventos.

Registrar actividad.

Consultar resultados.

Editar perfil.

11. Instrucciones para Ejecutar el Proyecto

11.1 Ejecución del Backend Local

Cada microservicio se ejecuta de manera independiente.

Ejemplo con Auth Service:

```
cd backend-auth
```

```
npm install
```

```
npm start
```

El mismo proceso aplica para:

- backend-user
- backend-events
- backend-registrations
- backend-results
- backend-activity

11.2 Variables de Entorno

En Render, las variables de entorno están configuradas directamente en la plataforma. Por esta razón, la aplicación desplegada funciona correctamente sin depender de archivos .env locales dentro del repositorio.

Para ejecución local, cada microservicio requiere variables similares a:

PORT
DB_HOST
DB_PORT
DB_USER
DB_PASSWORD
DB_NAME
JWT_SECRET

Algunos servicios también requieren URLs de otros microservicios, por ejemplo:

EVENTS_SERVICE_URL
USER_SERVICE_URL
REGISTRATIONS_SERVICE_URL

11.3 Ejecución de la Aplicación Android

Para ejecutar la aplicación móvil:

1. Abrir Android Studio.
2. Seleccionar la carpeta frontend-android/EventraMobile.
3. Sincronizar Gradle.
4. Ejecutar la aplicación en emulador o dispositivo físico.
5. Iniciar sesión con las credenciales de prueba.

La aplicación también cuenta con un APK incluido en el repositorio para instalación directa.

12. Bases de Datos

El proyecto utiliza PostgreSQL con bases de datos separadas por microservicio:

- eventra_auth
- eventra_user

- eventra_events
- eventra_registrations
- eventra_results
- eventra_activity

Los scripts SQL se encuentran dentro de la carpeta Script-SQL del repositorio.

13. Datos de Prueba

El proyecto cuenta con datos de prueba coherentes entre microservicios:

Usuarios de tipo organizador y corredor.

Eventos creados por el organizador.

Inscripciones asociadas a corredores.

Actividades deportivas.

Resultados publicados.

Ranking visible para eventos finalizados.

Esto permite evaluar el flujo completo de la aplicación sin necesidad de crear todos los datos manualmente.

14. Observaciones Finales

Eventra cumple con los requerimientos de la entrega final al incluir repositorio GitHub, servicios desplegados en Render, informe técnico, descripción de microservicios, endpoints, patrones de diseño, pruebas realizadas, credenciales, instrucciones y datos necesarios para la evaluación.

La aplicación fue desarrollada como una solución móvil Android para la gestión de eventos deportivos de running y se encuentra preparada para futuras mejoras como notificaciones avanzadas, estadísticas detalladas, reportes administrativos e integración con dispositivos deportivos.